

PYTHON SUMMER PLAN · DAY 22

tkinter basics

windows, widgets & the event loop

your code finally gets a face — a real window with buttons instead of a terminal prompt

Mon Jun 8, 2026 · 75 min · Yuvraj Ruparel

Week 3 closed: Python runs your Mac

The Morning Launcher (launcher.py v2) opens Safari tabs, starts Spotify, fires notifications — all JSON-configured. But everything still runs through a terminal: no window, no buttons, no place to type input while it runs.

BEFORE — Week 3 tools

- CLI entry point: `python3 launcher.py morning`
- Config lives in `routines.json` — edit by hand
- Autoclicker CPS is a variable in the source code
- Output = `print()` lines scrolling past
- Only you can use it — needs terminal comfort

AFTER — this week (tkinter)

- A real window: title bar, close button, resize
- Buttons trigger your functions on click
- Entry boxes take typed input while running
- Day 28: autoclicker gets a CPS slider + start/stop
- Same skills later draw the CRUX HUD panels

Today is deliberately small: one window, three widgets, one callback. Layout systems come tomorrow (Day 23).

Tk() makes a window, mainloop() keeps it alive

tkinter is Python's built-in GUI library — it draws windows and widgets. Every tkinter program starts the same way: create one root window object, configure it, then hand control to its event loop.

window_demo.py

```
import tkinter as tk

root = tk.Tk()      # the one main window
root.title("Hello tkinter")
root.geometry("360x160") # width x height in px

print("before mainloop")
root.mainloop()      # blocks here until window closes
print("after close")  # runs only AFTER you close it
```

OUTPUT

```
before mainloop
(window stays open... close it...)
after close
```

tk.Tk() — the root

One per program. It's the top-level window the OS manages: title bar, close button, the works.

mainloop() — the event loop

An infinite loop that waits for events (clicks, keys) and redraws the window. Your script pauses on this line; tkinter takes over.

Same idea as pygame

Week 2's while running: ... loop, but tkinter owns the loop — you just register what to do when events happen.

Widgets: created, then placed

A widget is any visible element inside the window — text, buttons, input boxes. A Label just displays text. Creating a widget is two separate steps: build it (parent + options), then place it with a geometry manager like pack().

label_demo.py

```
import tkinter as tk
```

```
root = tk.Tk()
root.title("Labels")
```

```
# step 1: create — parent first, options as kwargs
greeting = tk.Label(root, text="Hello, Yuvraj",
                    font=("Lato", 16))
```

```
# step 2: place — nothing shows until you do this
greeting.pack(pady=20)
```

```
root.mainloop()
```

Parent argument

Every widget's first argument is its container — here root. Widgets form a tree, window at the top.

create ≠ show

tk.Label(...) builds the object in memory. Forget .pack() and the window opens empty — the #1 beginner bug.

pack() for now

Simplest geometry manager. grid() comes Day 23.

TAKEAWAY

Two-step rule: construct the widget, then pack it. An invisible widget almost always means a missing pack/grid call.

command= wires a click to a function

A Button runs a function when clicked. You pass the function itself to `command=` — no parentheses. This is the callback pattern: you hand tkinter a function, and the event loop calls it later, on each click.

`button_demo.py`

```
import tkinter as tk

def on_click():
    print("clicked!")

root = tk.Tk()

btn = tk.Button(root, text="Press me",
                 command=on_click) # NO parentheses
btn.pack(pady=20)

root.mainloop()
```

OUTPUT

```
clicked!
clicked! # once per press
```

`command=on_click`

Passes the function object. Same idea as the ACTIONS dispatch dict in `launcher.py` — functions are values.

`command=on_click()`

With parentheses it runs immediately, once, at build time — and the button gets its return value (None). Classic bug.

Who calls it?

You never call `on_click` yourself. `mainloop()` detects the click event and invokes it for you.

Entry replaces input() for windowed apps

An Entry is a one-line text box the user can type into while the app runs. Unlike input(), it doesn't pause the program — you read its current contents with .get() whenever you need them, usually inside a button callback.

entry_demo.py

```
import tkinter as tk

def show():
    text = name_entry.get() # read what's typed NOW
    print(f"you typed: {text}")

root = tk.Tk()

name_entry = tk.Entry(root, width=28)
name_entry.pack(pady=10)
tk.Button(root, text="Read it", command=show).pack()

root.mainloop()
```

.get() returns a str

Always a string — even "42". Numeric tabs in the Day 27 calculator will need float(entry.get()) + error handling.

Read on demand

The entry doesn't push data to you. Your callback pulls the current value at click time.

.delete(0, tk.END)

Clears the box — handy after submitting. .insert(0, "text") pre-fills it.

OUTPUT

you typed: hello tkinter

Entry → Button → Label: the echo pattern

This is today's full deliverable in miniature: type into the Entry, click the Button, and the callback rewrites the Label with `config()`. Read input → process → update display is the skeleton of every GUI tool you'll build this week.

echo.py

```
import tkinter as tk

def echo():
    text = entry.get().strip()
    result.config(text=text or "(empty)")

root = tk.Tk(); root.title("Echo")

entry = tk.Entry(root, width=30)
button = tk.Button(root, text="Echo", command=echo)
result = tk.Label(root, text="...", font=("Lato", 14))

for w in (entry, button, result):
    w.pack(pady=8)

root.mainloop()
```

`widget.config(...)`

Changes options after creation — here the label's text. Any constructor kwarg can be updated this way.

State lives in widgets

No global string needed: the Entry holds the input, the Label holds the output. The callback just moves data between them.

Define before wiring

echo references entry and result, so the callback runs only after they exist. Defining functions at the top is safe — they're not called until clicked.

Four pieces, one pattern

Everything today reduces to this table. Memorize the construct → place → react cycle; every later widget (sliders, tabs, text areas) follows it.

Piece	What it is	Key call	Gotcha
<code>tk.Tk()</code>	The root window — one per app	<code>root.mainloop()</code> to start	Code after <code>mainloop</code> waits for close
<code>tk.Label</code>	Displays text or images	<code>lbl.config(text=...)</code> to update	Invisible until packed
<code>tk.Button</code>	Runs a callback on click	<code>command=func</code>	<code>command=func()</code> fires immediately
<code>tk.Entry</code>	One-line text input	<code>entry.get()</code> → str	Always returns str — cast for math

 TAKEAWAY

The mental model: you don't write the loop anymore. You build widgets, register callbacks, and `mainloop()` drives everything. Your code becomes event-driven.

hello_tk.py — your first window

Build a small window with an entry box, a button, and a label. Typing text and clicking the button echoes the entry's contents into the label. Exactly the slide-5 pattern — but typed from scratch, not copied.

STRUCTURE

- One file: `hello_tk.py`
- Imports, callback, widgets, `mainloop` — in that order
- Window title + geometry set
- Run: `python3 hello_tk.py`

WIDGETS

- `tk.Entry` — input box
- `tk.Button` — "Echo" label
- `tk.Label` — starts as "..."
- All placed with `pack(pady=...)`
- Bonus: `ttk` versions (Day 23 preview)

BEHAVIOR

- Click → label shows entry text
- Empty entry → label "(empty)"
- Bonus: clear entry after echo
- Bonus: bind `<Return>` so Enter key triggers the same callback

First: verify your install — run `python3 -m tkinter`. If a small test window opens, you're set. If import fails on a Homebrew/pyenv Python, you may need `python-tk` — I'm not certain which install you'll use, so check this first.

The four bugs everyone hits on day one

⚠️ **command=func, not func()**

Parentheses call the function at build time. The button then does nothing — its command is the return value, None. If your 'click' fires once at startup, this is why.

⚠️ **Don't block mainloop**

time.sleep() or a long while loop inside a callback freezes the whole window — the event loop can't redraw. Use root.after(ms, fn) for timed work (same rule as the Pomodoro plan).

💡 **Widget invisible? Check pack**

Creating a widget doesn't display it. Every widget needs a geometry call: pack(), grid(), or place(). Also: don't mix pack and grid in the same container.

💡 **Keep a widget reference**

If you'll update or read a widget later (entry.get(), label.config), assign it to a name. tk.Label(root, ...).pack() returns None — you can't touch it again.

One window today, a control panel by Sunday

DAY 23 · TOMORROW

tkinter layout

pack() got you here, but forms need grid(): rows, columns, Frame containers, and ttk themed widgets so things stop looking like 1995.

DAY 27-28 · CAPSTONE

Engineering calculator

Entry + Button + Label is literally one calculator tab: type F and A, click compute, $\sigma = F/A$ appears in a label. ttk.Notebook stacks four of these.

DAY 28 + JULY · GUI ERA

Autoclicker GUI & CRUX HUD

Sunday retrofits the autoclicker with a CPS slider and start/stop buttons. The same widget tree later becomes CRUX's HUD panels.

Three weeks of logic, one week of faces. The hard part — functions, state, dispatch — you already own. tkinter is just where it becomes visible.